

Tracking instrument tips through missing detections

Updated methodology, runnable implementation and measured results

Research question: Can Kalman tracking and short trajectory histories improve instrument-tip collision detection compared with the original 43-feature approach?

Measured answer

On this single-video experiment, Random Forest with Kalman trajectories and two seconds of history increases frame F1 from **0.319 to 0.348**. Missing-tip frame F1 increases from **0.448 to 0.504**. However, event recall falls from **0.547 to 0.406**. The evidence supports a modest frame-level benefit, not an overall improvement in event detection.

Data	Completed work
3917 sampled frames; 558 positive frames	Two independent Kalman filters; uncertainty and dropout age
64 collision annotation rows; one video	Six grouped model comparisons; inner-validation thresholds
24 temporal groups; five outer folds	Frame metrics, missing-tip analysis and one-to-one event matching

The original paper and its results remain available. This report uses a revised evaluation protocol and new experiments; the older headline scores are historical context only.

Reading guide: pages 2-7 explain each step; pages 8-10 report outcomes and limitations; pages 11-12 provide reproduction instructions and sources.

1. What changes in the existing project

Simple explanation

The detector supplies observations. A tracker estimates each tip position between observations and records how uncertain that estimate becomes. A classifier uses the resulting movement history to score collision likelihood. Neighboring positive scores are then grouped into events.

```
Original implementation
Video -> YOLO11n-seg -> 43 features -> classifier

Updated implementation
Cached YOLO detections -> separate TIPL/TIPR Kalman filters
-> trajectory + motion + missingness + uncertainty
-> current features or trailing 2-second summaries
-> Random Forest / XGBoost -> smoothed probability
-> threshold -> collision events
```

The original baseline already contains temporal information

The 43 features include constant-velocity extrapolation, rolling statistics and motion features. It is therefore not a pure single-frame baseline. The new comparison retains all 43 cached columns in Original43, then replaces the old tracking and temporal terms in the Kalman variants.

Variant	Feature count	Purpose
Original 43 / RF	43	Rerun existing features under revised evaluation
Kalman / RF	53	Current detector plus Kalman/motion/uncertainty features
Kalman + history / RF	85	Add trailing 2-second summaries

Each feature set is evaluated with both Random Forest and XGBoost. Neural temporal models are deferred: only one labeled video is available, so independent sequence diversity is limited. No new LSTM, GRU, TCN or Transformer result is claimed.

2. Step 1 - detector inputs and labels

Simple explanation

For each sampled frame, keep the highest-confidence detection of each instrument tip. The existing extraction script already supplies the required box centers, bounding boxes, confidence and mask-overlap measurements. Reusing these values holds detector behavior constant between methods.

Input	Format and meaning
data/video.mp4	Original video; detector inference can be reproduced with script 02.
models/video_tip_circle_yolo11n_seg_best.pt	Existing trained TIPL, TIPR and TIPandCircle detector.
data/features_and_labels.csv	One row per sampled frame; frame_idx, time_s, 43 features and three labels.
TIPL_* / TIPR_*	present, x, y, w, h, conf, area. Coordinates are in image pixels.
mask_iou_TIPL_TIPR	Saved overlap of the detected binary segmentation masks.
data/ground_truth.csv	start_time_s, end_time_s, is_collision, severity and near-miss fields.

The cache contains 3917 frames, with median sampling interval 0.196 s (about 5.10 Hz). Calculations use actual timestamps rather than assuming exactly 5 Hz.

```
center = ((x1 + x2)/2, (y1 + y2)/2)
box = (x1, y1, x2, y2)
mask_iou = count(mask_L & mask_R) / count(mask_L | mask_R)
y(t) = 1 if any collision interval covers t with +/-0.5 s padding
```

Masks themselves were not persisted in the original CSV. Their precomputed overlap is reused; new masks are not fabricated during occlusion. Missing overlap is disambiguated using detection-presence and tracking-state features. The box center is a tip-location proxy, not an annotated physical contact point.

The segmentation interface exposes per-instance masks, boxes and confidence scores [1]. To regenerate detections, use the existing script before running the new experiment; the completed run used the cache.

3. Step 2 - one Kalman filter per tip

Simple explanation

Each filter remembers position and velocity. It first predicts the next position. If YOLO sees the tip, the filter corrects the prediction using the observation. If the tip is missing, it predicts for at most 1.5 seconds. After that, the position is unavailable until a new detection initializes the track.

State: $s = [x, y, vx, vy]^T$ Observation: $z = [x, y]^T$

$F = \begin{bmatrix} 1, 0, dt, 0 \\ 0, 1, 0, dt \\ 0, 0, 1, 0 \\ 0, 0, 0, 1 \end{bmatrix}$

$H = \begin{bmatrix} 1, 0, 0, 0 \\ 0, 1, 0, 0 \end{bmatrix}$

$G = \begin{bmatrix} dt^2/2, 0 \\ 0, dt^2/2 \\ dt, 0 \\ 0, dt \end{bmatrix}$

$Q = 100^2 * G * G^T$

$R = (8^2 / \max(\text{YOLO_confidence}, 0.1)) * I_2$

Predict: $s_{\text{minus}} = F s$; $P_{\text{minus}} = F P F^T + Q$

Update: $K = P_{\text{minus}} H^T (H P_{\text{minus}} H^T + R)^{-1}$

$s = s_{\text{minus}} + K (z - H s_{\text{minus}})$

$A = I - K H$

$P = A P_{\text{minus}} A^T + K R K^T$

The Joseph covariance update maintains numerical stability. The code solves a linear system instead of explicitly inverting a matrix. Initial position variance is the measurement variance; initial velocity variance is 10,000 pixels squared per second squared.

Field	Interpretation
state = 0 / 1 / 2	YOLO-updated estimate / predicted estimate / unavailable
missing_frames and age_s	Consecutive missing sampled frames and seconds since a real detection
variance = Pxx + Pyy	Position covariance trace in pixels squared; larger means less certain
max_missing_s = 1.5	Expire old tracks and reinitialize on reacquisition

Process noise, measurement noise and the confidence-to-variance rule are fixed engineering assumptions, not calibrated detector error measurements. Identity follows the TIPL/TIPR class labels; identity swaps and false detections are not corrected by this filter.

4. Steps 3-4 - trajectories and features

Simple explanation

The position estimates form a trajectory for each tip. Useful collision signals include closeness, approach speed, sudden motion changes, overlap, and whether the estimate is based on a visible tip or a prediction.

```
T_L = [(x_L(t1), y_L(t1)), ..., (x_L(tn), y_L(tn))]  
T_R = [(x_R(t1), y_R(t1)), ..., (x_R(tn), y_R(tn))]
```

```
d(t) = sqrt((x_L-x_R)^2 + (y_L-y_R)^2)  
delta_d(t) = d(t) - d(t-1)  
distance_rate(t) = delta_d(t) / dt  
speed_tip(t) = sqrt(vx(t)^2 + vy(t)^2)  
acceleration_tip(t) = (speed(t) - speed(t-1)) / dt  
relative_speed(t) = norm(v_L(t) - v_R(t))
```

Negative distance rate indicates approach; positive distance rate indicates separation. Acceleration here is the derivative of speed, not a full vector-acceleration estimate. Missing positions stay unavailable in the trajectory export, and missing model inputs use a fixed -1 sentinel alongside state indicators.

Feature family	What it contributes
Detector geometry	Tip/marker centers, sizes, confidence, mask area, raw distances and overlap
Kalman state	Filtered x/y, vx/vy, speed and acceleration for both tips
Pair motion	Distance, distance difference/rate and relative speed
Occlusion proxies	Detection presence, track state, missing frame count, age and covariance trace
Temporal summaries	Mean, minimum, maximum and standard deviation over the trailing 2 seconds

Temporal summaries cover distance, distance rate, both speeds, both position variances and both state codes. The means of state codes combine prediction and loss severity; explicit missingness fields remain available. Rolling windows and tracker state reset at evaluation-group boundaries.

A missing detection is only a proxy for occlusion: it may also reflect detector error or a tip outside the field of view. Two-dimensional proximity cannot establish physical contact in depth.

5. Steps 5-6 - history and model training

Simple explanation

Two seconds of past motion lets a tree classifier use recent approach and separation patterns. This is temporal feature engineering; it is not a neural sequence model. A causal predictor cannot use separation that has not happened yet, so recognition may be delayed until later frames.

Setting	Random Forest	XGBoost
Estimators	300	250
Maximum depth	10	4
Regularization	Minimum leaf size 3	Minimum child weight 2
Imbalance handling	Balanced class weights	Training negatives / positives
Other	Seed 42	Learning rate .05; row/column sampling .9

Grouped evaluation and threshold selection

Five outer StratifiedGroupKFold splits use 24 contiguous groups. Candidate boundaries begin every 30 seconds and move beyond collision annotations plus a 4.5-second margin. Thus padded event windows are never divided between groups. Four seconds of nearby training samples are purged around held-out groups.

Inside each outer training set, the first split of a separate three-way grouped splitter forms inner fit and validation sets. The inner model predicts validation scores; a threshold from 0.05 to 0.95 in 0.01 steps maximizes validation F1. A fresh model then fits the full purged outer training set and applies that fixed threshold to the outer test set.

```
for outer_train, outer_test in grouped_splits:
    purge outer_train near outer_test
    inner_fit, inner_val = grouped_split(outer_train)
    purge inner_fit near inner_val
    fit inner_model; score and smooth inner_val
    threshold = argmax(validation_F1)
    refit model on outer_train
    score outer_test; smooth within each group
    save predictions using the validation threshold
```

All six variants use identical outer splits. No labels, timestamps, frame numbers, severity or near-miss labels enter the classifier feature matrix. Group membership remains disjoint between fit, validation and test sets [2].

6. Steps 7-8 - probability to events

Simple explanation

Average each score with up to two preceding scores, apply the validation-selected threshold, and join nearby positive samples into an event. A long burst should produce one event, not one event per frame.

```
p_smooth(t) = mean(p(t), p(t-1), p(t-2))  
y_hat(t) = 1[p_smooth(t) >= threshold_from_inner_validation]
```

Within each evaluation group:
collect positive samples in time order
merge if next positive time - previous positive time <= 0.4 s
start = first positive timestamp
end = last positive timestamp
peak = highest smoothed score within the event
confidence = score at peak
duration = end - start

At the observed sampling interval, a 0.4-second positive-to-positive gap can bridge one intervening negative sample. Smoothing and merging never cross group boundaries. Single-sample events are retained with duration zero. Event end and peak are finalized after the event closes.

Example required output

start_frame	end_frame	peak_s	confidence	duration_s
105	110	Timestamp of peak	Maximum score	t(110) - t(105)

This row illustrates the format only. Actual events are saved for every model in results/kalman_updated/*_events.csv with group, start_frame, end_frame, start_s, end_s, peak_s, confidence and duration_s.

Event matching

A prediction is eligible if it overlaps a human interval expanded by 0.5 seconds. Maximum-cardinality one-to-one assignment prevents one long prediction from counting as several detected collisions. Unmatched predictions are false events; unmatched annotations are misses. A small midpoint-distance tie-break resolves equally sized assignments.

All 64 collision annotation rows are counted separately, including touching or overlapping rows. This convention penalizes a merged prediction spanning several annotations. Strict zero-tolerance results are also saved. Peak timing is compared with the annotation midpoint because a human peak-contact time is unavailable.

7. Measured frame-level results

These are pooled held-out predictions from the new protocol. Thresholds vary by outer fold and are selected only on inner validation data. RF means Random Forest; XGB means XGBoost.

Method	Precision	Recall	F1	ROC-AUC	Avg. prec.
Original 43 / RF	0.219	0.591	0.319	0.694	0.256
Original 43 / XGB	0.195	0.616	0.296	0.614	0.206
Kalman / RF	0.177	0.659	0.280	0.696	0.271
Kalman / XGB	0.198	0.591	0.297	0.621	0.206
Kalman + history / RF	0.232	0.695	0.348	0.706	0.294
Kalman + history / XGB	0.226	0.602	0.328	0.661	0.272

Confusion matrices

Method	TN	FP	FN	TP
Original 43 / RF	2180	1179	228	330
Original 43 / XGB	1938	1421	214	344
Kalman / RF	1652	1707	190	368
Kalman / XGB	2023	1336	228	330
Kalman + history / RF	2074	1285	170	388
Kalman + history / XGB	2206	1153	222	336

For RF, Kalman + history changes F1 by +0.029, ROC-AUC by +0.012, and average precision by +0.039. Kalman alone reduces F1 despite a similar ROC-AUC. Threshold choice and temporal features affect the practical tradeoff.

Why these scores differ from the old paper

Historical RF ROC-AUC was approximately 0.857 and XGBoost F1 approximately 0.528. Those scores used smaller groups, centered smoothing and a threshold chosen from pooled out-of-fold labels. The new run uses larger event-preserving groups, guarded splits, causal smoothing and inner-validation thresholds, with new fixed training settings. Historical and new scores are not a like-for-like measure of improvement.

8. Missing-tip and event-level results

Method	Missing-tip F1	Visible-tip F1	Missing-tip recall
Original 43 / RF	0.448	0.205	0.692
Original 43 / XGB	0.446	0.164	0.771
Kalman / RF	0.448	0.161	0.775
Kalman / XGB	0.454	0.164	0.733
Kalman + history / RF	0.504	0.211	0.835
Kalman + history / XGB	0.474	0.191	0.746

The missing-tip subset has 1008 frames, including 315 positive frames. It is defined by at least one missing YOLO tip. The RF history variant improves F1 in this subset, but direct occlusion annotations would be needed to isolate true occlusion performance.

Method	Detected	Missed	False	Event F1	Recall
Original 43 / RF	35	29	83	0.385	0.547
Original 43 / XGB	31	33	99	0.320	0.484
Kalman / RF	24	40	89	0.271	0.375
Kalman / XGB	25	39	86	0.286	0.391
Kalman + history / RF	26	38	62	0.342	0.406
Kalman + history / XGB	23	41	80	0.275	0.359

Method	Start MAE (s)	End MAE (s)	Peak/midpoint MAE (s)
Original 43 / RF	1.662	1.785	0.473
Original 43 / XGB	2.642	2.089	0.680
Kalman / RF	3.286	3.469	0.554
Kalman / XGB	2.737	2.440	0.943
Kalman + history / RF	2.886	2.936	0.829
Kalman + history / XGB	1.836	3.676	0.579

Timing errors are calculated on matched pairs only. A low timing error does not compensate for missed events. Broad predicted bursts can overlap an annotation even when their start and end times are inaccurate; duration and event recall should therefore be reviewed together.

9. Interpretation and next experiments

What the completed experiment establishes

The RF history variant detects 26 of 64 annotations versus 35 for the original-feature RF, while reducing false events from 83 to 62. It improves per-frame classification but merges or misses more individual annotations. It should not replace the baseline on an event-recall objective without further work.

What remains uncertain

This is a single-video pilot, not evidence of generalization across trainees or cameras. The detector training/test provenance has not been audited. Missing detections are not verified occlusions, and 2D box centers do not measure physical tip contact. The original cached feature history was computed globally, while new tracks reset by group; the guard reduces local overlap but does not make both history implementations identical.

Parameters were fixed for this run. The temporal ablation changes both tracking and feature representation; it does not isolate the benefit of a Kalman filter from adding more history features. Scores are uncalibrated classifier outputs, so an event confidence of 0.8 should not be interpreted as a validated 80% probability.

Prioritized follow-up experiments

Experiment	Question and procedure
More independent videos	Use held-out videos/trainees; audit detector training overlap before testing.
Matched history ablation	Compare no tracking, constant velocity and Kalman with identical causal histories and resets.
Tracker sensitivity	Tune Q, R and expiry (0.5/1.0/1.5 s) only inside training-side validation.
Event tuning	Choose smoothing, hysteresis, gap and minimum duration using validation event F1, then evaluate once.
Occlusion annotations	Mark true hidden-tip intervals and duration bins; evaluate tracking error with labeled positions.
Neural temporal models	With sufficient independent sequences, test small GRU/TCN first; keep complete windows within groups.

Report paired group/video bootstrap uncertainty in a larger study. Do not treat thousands of neighboring frames as independent samples. Resolve whether touching annotation rows represent separate contacts before standardizing event metrics.

10. Reproduce the work step by step

Run these commands from the project root. The completed experiment reused the existing detector cache and left the original paper and results intact.

1. Optional: regenerate the detector cache

```
venv\Scripts\python.exe scripts\02_extract_features.py
# Default input: data/video.mp4; default sampling: about 5 Hz
# This command overwrites data/features_and_labels.csv.
```

2. Check tracking and event logic

```
venv\Scripts\python.exe scripts\test_kalman_experiment.py
```

Four deterministic tests pass: expiry/reacquisition, constant-velocity prediction and positive covariance, one-to-one matching, and group boundaries for smoothing/event extraction.

3. Run all six comparisons

```
venv\Scripts\python.exe scripts\15_kalman_experiment.py
```

This writes trajectories.csv, metrics.json, protocol.json and six prediction/event CSV pairs to results/kalman_updated. It fits models for evaluation; it does not export a single production classifier. The report reports cross-validation results rather than training-set scores.

4. Build the PDF

```
python scripts\16_generate_updated_pdf.py
```

Use a Python environment with ReportLab for the PDF builder. The experiment uses NumPy, pandas, SciPy, scikit-learn and XGBoost from the project environment. The PDF reads measured JSON results; it does not invent performance values.

Traceability

The protocol records group boundaries, parameters, sample counts and the input CSV SHA-256. The feature lists and fold-level metrics/thresholds are recorded for every model. Saved predictions include time, original frame, group, outer fold, ground truth, probability, binary prediction and missing-tip status.

Input CSV SHA-256:
664f1c3ae18febed4a9572210220f762e8cf7d0732b062da53d69c53c686a8b2

11. Implementation map and references

Python function	Responsibility
TipKalman.step	Predict/correct a tip state; expire stale tracks; expose uncertainty.
make_groups	Move temporal boundaries outside padded collision intervals and history margins.
features	Build per-tip trajectories, pair motion and causal 2-second summaries.
model / main	Fit RF/XGB with grouped inner validation and five outer test folds.
extract_events	Join positive samples within groups and export event timing/confidence.
event_metrics	One-to-one event assignment and matched timing errors.

Minimal tracker usage

```
# TipKalman is defined in scripts/15_kalman_experiment.py
left = TipKalman(max_missing=1.5)
right = TipKalman(max_missing=1.5)

# One update per sampled timestamp:
L = left.step(0.0, [120.0, 80.0], confidence=0.9)
R = right.step(0.0, [170.0, 85.0], confidence=0.8)
L_next = left.step(0.2, measurement=None)
# [x, y, vx, vy, state, missing_frames, age_s, variance]
```

Sources and project evidence

[1] Ultralytics, Instance Segmentation and Prediction documentation. Per-instance boxes and masks are available through Results. docs.ultralytics.com/tasks/segment and docs.ultralytics.com/modes/predict. Accessed September 2026.

[2] scikit-learn, Cross-validation: evaluating estimator performance. Grouped evaluation keeps a group out of both train and test simultaneously. scikit-learn.org/stable/modules/cross_validation.html. Accessed September 2026.

[3] Project sources: scripts/02_extract_features.py; scripts/model_cv_utils.py; configs/experiment_config.yaml; paper/source/collision_detection_paper.md; data/ground_truth.csv; data/features_and_labels.csv.

[4] New empirical evidence: results/kalman_updated/metrics.json, protocol.json, trajectories.csv and per-model prediction/event exports. Full Python implementation: scripts/15_kalman_experiment.py. All numerical results in this report are loaded from these saved outputs.